Petmor Sistema Veterinaria Spa

4. Refactorizacion aplicando DRY

Problema: codigo repetido

En el flujo inicial, cada controlador podia validar las fechas por separado:

```
// Antes: repetido en Ventas, Gastos y Reportes
if (fechaInicio > fechaFin)
{
    ModelState.AddModelError("", "La fecha inicial no puede ser posterior.");
    return View(model);
}
```

Si la regla cambiaba, habia que corregirla en varias acciones.

Despues: una sola regla reutilizable

La regla se extrajo a `Infrastructure/FiltroFechas.cs` y se registra una vez:

```
// Program.cs
options.Filters.Add(new Petmor.Web.Infrastructure.FiltroFechas());

// FiltroFechas.cs
if (fechas.GetValueOrDefault(inicio) is DateTime d &&
    fechas.GetValueOrDefault(fin) is DateTime h && d.Date > h.Date)
    error = "La fecha inicial no puede ser posterior a la fecha final.";
```

Tambien se centralizaron las reglas de venta en `OperacionVentaService`, las reservas en `ReservaService`, las lecturas en `CodigosLecturas` y los recargos en `PagoReglas`.

```
flowchart TB
    A[Reglas repetidas] --> B[Extraer reglas comunes]
    B --> C[Servicios y helpers reutilizables]
    C --> D[Controllers delgados]
    C --> E[Pruebas unicas para cada regla]
```

Resultado

La regla se mantiene en un solo lugar, se reduce el codigo duplicado y las pruebas pueden validar el comportamiento sin levantar toda la interfaz.

```
flowchart LR
    A[Validaciones repetidas en Controllers] --> B[Extraer regla comun]
    B --> C[FiltroFechas y servicios reutilizables]
    C --> D[Controllers delgados]
    C --> E[Pruebas unicas para la regla]
```